

Software Engineer - Full Stack [Take Home Test]

Forethought

We believe that the true essence of software engineering extends far beyond algorithmic problem-solving. In crafting this take-home exercise, we've thoughtfully designed it to reflect the authentic challenges and experiences a software engineer would encounter in a day-to-day environment.

As practicing software engineers ourselves, we understand that no one can be an expert in every technology or framework. The reality of our profession is that we constantly navigate new territories, learn unfamiliar tools, and solve complex problems through research, experimentation, and collaboration. This exercise is designed to honor that reality.

We embrace the modern software development landscape where AI-powered tools like GitHub Copilot, ChatGPT, and Claude Code have become valuable allies in our development process. We strongly encourage you to leverage these tools, just as you would in your daily work (our team uses Claude Code day in day out). Your ability to effectively research, learn, implement, and deliver solutions matters more to us than any pre-existing knowledge.

The exercise deliberately spans multiple areas of software development – from backend services to frontend interfaces, from real-time data handling to testing strategies and deployment to AI/LLM Integration. This breadth isn't meant to overwhelm, but rather to give you the space to demonstrate your problem-solving approach and engineering mindset. We're interested in seeing how you make architectural decisions, handle trade-offs, and document your thinking process – rather than seeing the perfect solution.

When evaluating your submission, we'll focus on your engineering approach – how you structure your solution, the clarity of your code, the thoughtfulness of your documentation, and the reasoning behind your technical decisions. We care about seeing your authentic problem-solving process rather than a perfect implementation.

Take this opportunity to show us how you think, learn, and create as a software engineer. Document your journey, explain your choices, and don't hesitate to make pragmatic decisions about where to focus your efforts. We're looking forward to understanding your unique perspective and approach to software engineering through this exercise.

Most importantly, we're not looking for a perfect solution rather we wanna see how you think!

Overview

Design and implement a full-stack application that monitors HTTP responses and displays them in real-time. This assignment tests your ability to work with APIs, handle periodic tasks, manage databases, and create real-time web applications.

Core Requirements (Estimated Time - 4-6 hours)

Backend

1. Create a service that pings httpbin.org/anything endpoint every 5 minutes
2. Generate random JSON payload data for each request to send along with the request
3. Store the response data in a database of your choice
4. Broadcast new data to connected clients (web application)
5. Create REST endpoints to fetch historical data

Frontend

1. Create a simple dashboard (responsive design is a plus)
2. Display response data in a tabular format
3. Implement real-time updates when new data is received

Testing & CI Pipeline

1. Implement a CI pipeline using GitHub Actions, GitLab CI, or similar
 2. The pipeline should:
 - Run a test suite
 - Perform linting checks
 - Generate test coverage reports
 3. Focus testing on core functionality:
 - Choose and document what you consider core parts of the application
 - Write comprehensive tests for ONE of those core components
 4. Probable test categories:
 - Unit tests for critical business logic
 - Integration tests for key API endpoints
 - Basic end-to-end tests for critical user flows
-

AI Enhancement Challenge (Choose ONE, Est Time 2-4 hours)

To showcase your ability to work with modern AI technologies, you may optionally implement ONE of the following enhancements. **This is entirely optional but strongly encouraged** as it demonstrates skills highly relevant to our work

Option A: Anomaly Detection System

Build intelligent monitoring that learns patterns and detects issues automatically.

Requirements

- Calculate rolling statistics (mean, standard deviation) over time windows (1-hour minimum, 24-hour recommended)
- Detect anomalies using statistical methods (z-score, thresholds, or similar)
- Implement simple time-series forecasting to predict next expected response time
- Trigger alerts when anomalies or prediction errors exceed thresholds
- Visualize: real-time data, rolling averages, predicted values, anomaly markers, and confidence bands

What We're Looking For

- Statistical correctness and practical threshold tuning
 - Efficient real-time calculations that don't block the UI
 - Clear visualization of normal vs. anomalous behavior
 - Documentation of algorithm choices and trade-offs
-

Option B: LLM-Powered Insights

Add natural language analysis and automated incident reporting using LLMs.

Requirements

1. Natural Language Query Interface

- Chat widget where users ask questions about monitoring data:
 - "What were the slowest response times today?"
 - "Summarize any issues in the last 24 hours"
 - "Why did response time spike at 2pm?"
- System constructs appropriate database queries based on natural language
- LLM formats results in conversational responses

2. Automatic Incident Reporting

- When anomalies are detected (response time > 2x average):
 - Generate incident report automatically
 - Include timestamp, severity, affected endpoint
 - LLM suggests potential root causes
 - Create actionable recommendations
- Store incident reports in database
- Display in dedicated "Incidents" tab

3. Smart Response Analysis

- Analyze httpbin.org response payloads
- Extract and categorize key information
- Identify patterns in response data
- Generate natural language summaries

4. Cost Optimization (Critical!)

- Implement token counting before API calls
- Cache frequent queries and responses
- Rate limiting: max 20 LLM calls per hour
- Fallback behavior when quota exceeded
- Display cost estimation in dashboard
- **Document your cost analysis in detail**

What We're Looking For

- Cost consciousness
- Quality prompt engineering
- Graceful error handling and degradation
- Smart caching strategies
- Excellent user experience with streaming responses
- Token usage tracking and optimization

Option C: Intelligent Payload Analysis

Build a system that learns from response patterns and enables semantic search.

Requirements

1. Payload Pattern Recognition

- Analyze response structures from httpbin.org
- Extract common fields and patterns

- Categorize response types (success, error, slow, fast)
- Build searchable index of response patterns

2. Automated Tagging System

- Generate tags for each response using LLM or rule-based classification:
 - Content type: json, html, xml
 - Size category: small (<1KB), medium, large (>10KB)
 - Complexity: simple, nested, complex
- Store tags with each response record
- Enable filtering by tags in dashboard

3. Smart Search Interface

- Natural language search: "Find all slow JSON responses"
- Semantic search across response content
- Filter by multiple dimensions (time + tags + content)
- Highlight matching content in results

4. Pattern-Based Insights

- Identify recurring patterns in payloads
- Detect unusual payload structures
- Generate alerts for unexpected formats
- Trend analysis on payload characteristics

What We're Looking For

- Practical classification logic (LLM-based or rule-based)
- Effective search functionality
- Smart pattern detection
- Efficient data processing at scale

Technical Specifications

Backend Requirements

- **Preferred:** Node.js with Express.js, Nest.js, or similar framework
- **Alternative:** Python-based frameworks (FastAPI, Flask, Django)
- Choose any SQL or NoSQL database (justify your choice)
- Proper error handling and logging is a huge plus

Frontend Requirements

- **Preferred:** React or Next.js
- **Alternative:** Vue.js or Angular
- Implement proper state management
- Handling loading and error states is a huge plus
- Basic component tests are a huge plus

Infrastructure & Deployment

- Deploy to any free platform (Vercel, Netlify, Railway, Render, Heroku, etc.)
- Include environment variable configuration
- Document deployment process

Expected Deliverables

1. Source code in a Git repository
2. README.md with:
 - Setup instructions
 - Architecture overview
 - Choice of technologies and reasoning
 - Assumptions made
 - Future improvements
 - Testing strategy and core component identification
3. Database schema (if applicable)
4. Deployment of the solution is any Free platforms

Evaluation Criteria

- Proper execution of functional requirements
- Clarity & readability of code
- Thought process & decision-making process

Submission

- Share a GitHub/GitLab repository link to Asif Bin Hossain , Md Mizanur Rahman & Zahidul Hossain Choyan
- Ensure the repository is public or provide access to the above emails if private

Some words of wisdom

- Focus on code quality & pragmatic engineering rather than the perfect solution
- Some parts are intentionally kept vague for you to make appropriate assumptions, so don't be scared to make assumptions on any parts of the system but be rational about it.
- Document any assumptions or shortcuts taken due to time constraints.
- Feel free to use any additional libraries that you think are necessary
- Include instructions for running the application locally
- Clearly document your reasoning for identifying core components and testing priorities
- Be smart & don't be afraid to go outside of your comfort zone.
- If you have any questions or concerns, feel free to reach out to Asif Bin Hossain .

Good Luck, See you at the finish line!

